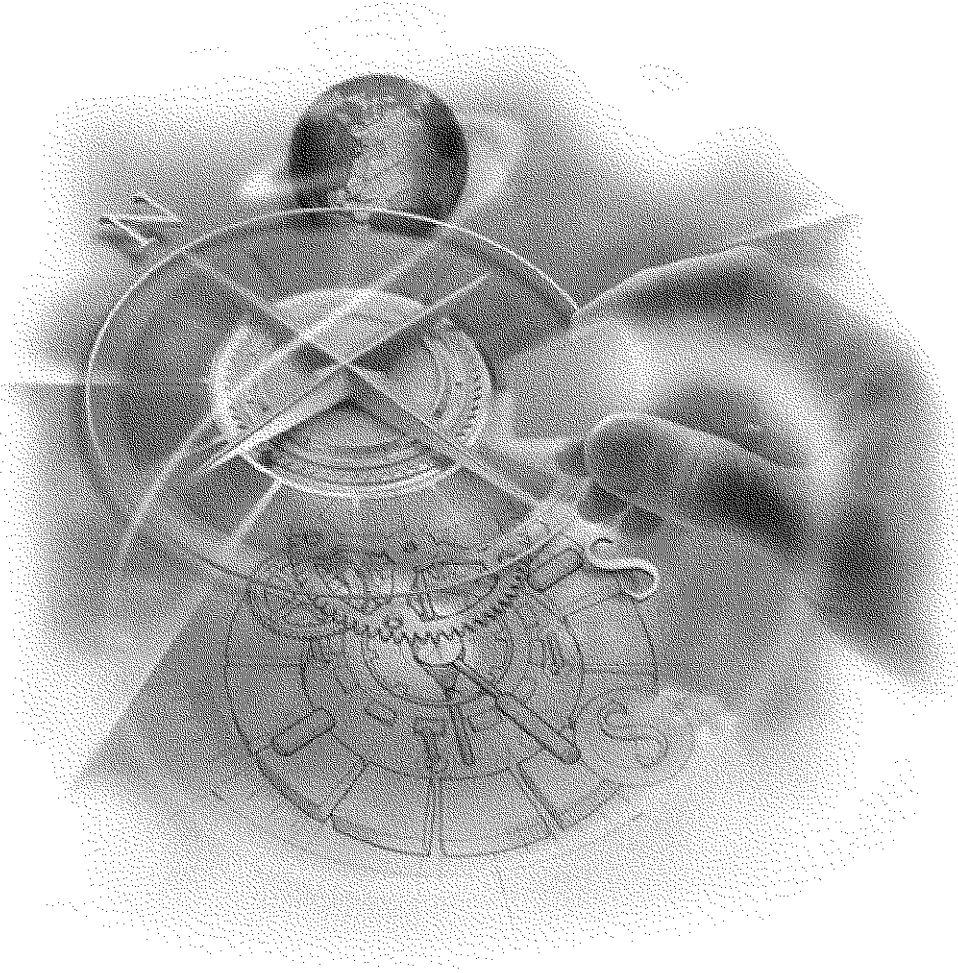


LDAP and NDS



Novell Developer Kit

Novell®

Legal Notices

Novell, Inc. makes no representations or warranties with respect to the contents or use of this documentation, and specifically disclaims any express or implied warranties of merchantability or fitness for any particular purpose. Further, Novell, Inc. reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes.

Further, Novell, Inc. makes no representations or warranties with respect to any software, and specifically disclaims any express or implied warranties of merchantability or fitness for any particular purpose. Further, Novell, Inc. reserves the right to make changes to any and all parts of Novell software, at any time, without any obligation to notify any person or entity of such changes.

This product may require export authorization from the U.S. Department of Commerce prior to exporting from the U.S. or Canada.

Copyright © 1993-2001 Novell, Inc. All rights reserved. No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of the publisher.

U.S. Patent Nos 5,553,139; 5,553,143; 5,677,851; 5,758,069; 5,784,560; 5,818,936; 5,864,865; 5,903,650; 5,905,860; 5,910,803 and other Patents Pending.

Novell, Inc.
122 East 1700 South
Provo, UT 84606
U.S.A.

www.novell.com

LDAP and NDS
September 2001
No Part Number Assigned

Online Documentation: To access the online documentation for this and other Novell developer products, and to get updates, see developer.novell.com/ndk. To access online documentation for Novell products, see www.novell.com/documentation.

Novell Trademarks

AppNotes is a registered trademark of Novell, Inc.

AppTester is a trademark of Novell, Inc., in the United States.

ArcNet 68 is a trademark of Novell, Inc.

ASM is a trademark of Novell, Inc.

BorderManager is a trademark of Novell, Inc.

BorderManager FastCache is a trademark of Novell, Inc.

BrainShare is a registered service mark of Novell, Inc., in the United States and other countries.

C3PO is a trademark of Novell, Inc.

Client 32 is a trademark of Novell, Inc.

ConsoleOne is a trademark of Novell, Inc.

Controlled Access Printer is a trademark of Novell, Inc.

Custom 3rd-Party Object is a trademark of Novell, Inc.

DENIM is a service mark of Novell, Inc.

DeveloperNet is a registered trademark of Novell, Inc. in the United States and other countries.

DeveloperNet 2000 is a trademark of Novell, Inc.

digitalme is a trademark of Novell, Inc.

DirXML is a trademark of Novell, Inc.

eDirectory is a trademark of Novell, Inc.

Full Service Directory is a trademark of Novell, Inc.

GroupWise is a registered trademark of Novell, Inc. in the United States and other countries.

Hardware Specific Module is a trademark of Novell, Inc.

HostPublisher is a trademark of Novell, Inc.

Hot Fix is a trademark of Novell, Inc.

HSM is a trademark of Novell, Inc.

iChain is a trademark of Novell, Inc.

instantme is a trademark of Novell, Inc.

Internetwork Packet Exchange is a trademark of Novell, Inc.

IPX is a trademark of Novell, Inc.

IPX/SPX is a trademark of Novell, Inc.

LANalyzer is a registered trademark of Novell, Inc. in the United States and other countries.

Link Support Layer is a trademark of Novell, Inc.

LSL is a trademark of Novell, Inc.

ManageWise is a registered trademark of Novell, Inc., in the United States and other countries.

Mirrored Server Link is a trademark of Novell, Inc.

MLI is a trademark of Novell, Inc.

MLID is a trademark of Novell, Inc.

MSL is a trademark of Novell, Inc.

Multiple Link Interface is a trademark of Novell, Inc.

Multiple Link Interface Driver is a trademark of Novell, Inc.

NControl is a trademark of Novell, Inc.

NCP is a trademark of Novell, Inc.

NDebug is a trademark of Novell, Inc.

NDPS is a registered trademark of Novell, Inc.

NDR is a trademark of Novell, Inc.

NDS is a registered trademark of Novell, Inc. in the United States and other countries.

NDS Administrator is a trademark of Novell, Inc.

NDS Manager is a trademark of Novell, Inc.

NE2000 is a trademark of Novell, Inc.

NetWare is a registered trademark of Novell, Inc. in the United States and other countries.

NetWare/IP is a trademark of Novell, Inc.

NetWare 386 is a trademark of Novell, Inc.

NetWare Aware is a trademark of Novell, Inc.

NetWare Connect is a registered trademark of Novell, Inc. in the United States.

NetWare Core Protocol is a trademark of Novell, Inc.

NetWare DOS Requester is a trademark of Novell, Inc.

NetWare Loadable Module is a trademark of Novell, Inc.

NetWare Management Portal is a trademark of Novell, Inc.

NetWare Management System is a trademark of Novell, Inc.

NetWare MHS is a registered trademark of Novell, Inc.

NetWare Name Service is a trademark of Novell, Inc.

NetWare Peripheral Architecture is a trademark of Novell, Inc.

NetWare Print Server is a trademark of Novell, Inc.

NetWare Requester is a trademark of Novell, Inc.

NetWare SFT and NetWare SFT III are trademarks of Novell, Inc.

NetWare SQL is a trademark of Novell, Inc.

NetWare Telephony Services is a trademark of Novell, Inc.

NetWare Tools is a trademark of Novell, Inc.

NetWare Upgrade is a service mark of Novell, Inc.

NIMS is a trademark of Novell, Inc.

NLM is a trademark of Novell, Inc.

NMAS is a trademark of Novell, Inc.

NMS is a trademark of Novell, Inc.

Novell is a registered trademark of Novell, Inc. in the United States and other countries.

Novell Application Launcher is a trademark of Novell, Inc.

Novell Authorized Reseller is a service mark of Novell, Inc.

Novell Authorized Service Center is a service mark of Novell, Inc.

Novell BorderManager is a trademark of Novell, Inc.

Novell Certificate Server is a trademark of Novell, Inc.

Novell Client is a trademark of Novell, Inc.

Novell Directory Services is a registered trademark of Novell, Inc.

Novell Distributed Print Services is a trademark of Novell, Inc.

Novell Embedded Systems Technology is a registered trademark of Novell, Inc., in the United States and other countries.

Novell HostPublisher is a trademark of Novell, Inc.

Novell Internet Caching System is a trademark of Novell, Inc.

Novell Labs is a trademark of Novell, Inc.

Novell SecretStore is a trademark of Novell, Inc.

Novell Security Attributes is a trademark of Novell, Inc.
Novell Storage Services is a trademark of Novell, Inc.
Novell Web Server is a trademark of Novell, Inc.
Novell, Yes, Tested & Approved logo is a trademark of Novell, Inc.
ODI is a trademark of Novell, Inc.
OnDemand is a service mark of Novell, Inc.
Open Data-Link Interface is a trademark of Novell, Inc.
Packet Burst is a trademark of Novell, Inc.
Personal NetWare is a trademark of Novell, Inc.
Printer Agent is a trademark of Novell, Inc.
Public Access Printer is a trademark of Novell, Inc.
QuickFinder is a trademark of Novell, Inc.
Red Box is a trademark of Novell, Inc.
Remote Console is a trademark of Novell, Inc.
RX-Net is a trademark of Novell, Inc.
Sequenced Packet Exchange is a trademark of Novell, Inc.
SFT, SFT III, and SFT NetWare are trademarks of Novell, Inc.
SMS is a trademark of Novell, Inc.
SMSTSA is a trademark of Novell, Inc.
SoftSolutions is a registered trademark of SoftSolutions Technology Corporation, a wholly owned subsidiary of Novell, Inc.
SPX is a trademark of Novell, Inc.
Storage Management Services is a trademark of Novell, Inc.
SVR4 is a trademark of Novell, Inc.
System V is a trademark of Novell, Inc.
Topology Specific Module is a trademark of Novell, Inc.
Transaction Tracking System is a trademark of Novell, Inc.
TSM is a trademark of Novell, Inc.
TTS is a trademark of Novell, Inc.
Universal Component System is a registered trademark of Novell, Inc.
ViewMAX is a trademark of Novell, Inc.
ZENworks is a trademark of Novell, Inc.

Third-Party Trademarks

All third-party trademarks are the property of their respective owners.
Java is a trademark or registered trademark of Sun Microsystems, Inc., in the United States and other countries.

Contents

Preface	9
1 LDAP and NDS Integration	11
Supported Platforms	12
eCommerce Beans	12
LDAP Libraries for C	12
LDAP Class Libraries for Java	13
LDAP Extensions and Controls for JNDI	13
Novell LDAP JDBC Driver	13
Novell Controls for ActiveX	13
LDAP and NDS Terminology.	13
Objects and Entries, Attributes and Properties	14
Naming Conventions	14
LDAP Naming Contexts and NDS Partitions	15
Access Control and Effective Rights	16
Containment in Flat and Hierarchical Directories.	16
Novell LDAP Servers.	16
LDAP Support for NDS Features.	18
LDAP Controls	19
Server-Side Sorting and Sort Keys	20
Virtual List Views	21
LDAP Extensions.	22
Differences Among LDAP Servers	24
Schema.	24
Validation and Security Checking	25
Directory Structure	26
LDAP Control Support	27
LDAP Extension Support	28
LDAP Search Filters	28
Connecting and Authenticating to NDS with LDAP	29
Connections on Port 389	30
SSL Connections on Port 636	30
Anonymous Binds	30
userPassword Attribute	30
LDAP and NDS Schema	31
Schema Naming Rules	32
Schema Mapping	32
LDAP Operational Attributes	33
Root DSE Attributes	35
Subschema Subentry Attributes	36

Attribute Flags	37
Object Class Flags.	40
Syntax Definitions	42
Auxiliary Classes versus Modifications to Class Definitions	44
Other Sources of LDAP Information.	44

2 LDAP Return Codes 45

LDAP Client Return Codes	45
LDAP Server Return Codes	47
LDAP Result Code Structure	51

Revision History 53

Preface

This book describes the following NDS and LDAP integration features:

- ♦ Terminology differences between NDS and LDAP
- ♦ LDAP controls and extensions for accessing NDS features
- ♦ NDS support for the LDAP schema
- ♦ LDAP support for NDS classes and attributes
- ♦ Authenticating to NDS through LDAP
- ♦ LDAP return codes common to all LDAP implementations

1

LDAP and NDS Integration

An NDS tree or directory can be accessed either through NDAP/NCP or LDAP. This document exposes the specific details you need to know to use LDAP to access NDS. It describes details about Novell's LDAP server and NDS directory. It assumes that you are familiar with the following:

- ♦ **LDAP v2 and v3.** If you are new to LDAP and need to develop a basic understanding of LDAP and the standard features of LDAP v3, see "Other Sources of LDAP Information" on page 44.
- ♦ **NDS.** If you are new to NDS and need to develop an understanding of basic NDS functionality, see *NDS Technical Overview*).

This document provides an overview of the following topics:

- ♦ "Supported Platforms" on page 12
- ♦ "LDAP and NDS Terminology" on page 13
- ♦ "Novell LDAP Servers" on page 16
- ♦ "LDAP Support for NDS Features" on page 18
- ♦ "LDAP Controls" on page 19
- ♦ "Differences Among LDAP Servers" on page 24
- ♦ "LDAP Search Filters" on page 28
- ♦ "Connecting and Authenticating to NDS with LDAP" on page 29
- ♦ "LDAP and NDS Schema" on page 31
- ♦ "Other Sources of LDAP Information" on page 44

Supported Platforms

Novell currently supplies the following LDAP development tools:

- ♦ eCommerce Beans (<http://developer.novell.com/ndk/ecb.htm>)
- ♦ LDAP Libraries for C (<http://developer.novell.com/ndk/cldap.htm>)
- ♦ LDAP Class Libraries for Java (<http://developer.novell.com/ndk/jldap.htm>)
- ♦ LDAP Extensions and Controls for JNDI (<http://developer.novell.com/ndk/extjndi.htm>)
- ♦ Novell LDAP JDBC Driver (<http://developer.novell.com/ndk/ldapjdbc.htm>)
- ♦ Novell Controls for ActiveX (<http://developer.novell.com/ndk/ocx.htm>)

The Novell LDAP server is currently available on NetWare 5.x, NDS for NT, NDS for Solaris, and NDS eDirectory (for NetWare, NT, Solaris, and Linux).

eCommerce Beans

The eCommerce beans provide easy-to-use Java components to integrate Web applications using LDAP.

LDAP Libraries for C

The LDAP Libraries for C kit enables application developers to write applications to access, manage, update, and search for information stored in NDS and other LDAP-aware directories. It currently supports development on the following NDS server platforms:

- ♦ NetWare 5.1
- ♦ NetWare 5 with SP4 or higher
- ♦ Windows NT server 4.0 with SP 4 or higher
- ♦ Solaris 2.6 and 2.7
- ♦ Linux (tested on RedHat 6.1, 6.2, and 7.0)

It supports development on the following client platforms:

- ♦ Windows NT workstation 4.0 with SP 3 and SP 4
- ♦ Windows 95

- ♦ Windows 98
- ♦ Windows 2000
- ♦ Solaris 2.6 and 2.7
- ♦ Linux (tested on RedHat 6.1, 6.2, and 7.0)

LDAP Class Libraries for Java

The LDAP Class Libraries for Java kit enables application developers to write applications to access, manage, update, and search for information stored in NDS and other LDAP-aware directories. It provides LDAP extensions for accessing and managing NDS-specific information such as effective rights, replicas, and partitions.

LDAP Extensions and Controls for JNDI

The LDAP Extensions and Controls for JNDI are designed to be used in applications that run on any operating system that has a JVM (Java virtual machine). The provider comes from Sun. Although other platforms are possible, this provider has been tested against the Novell LDAP server and NDS directory from the following platforms:

- ♦ NetWare
- ♦ Windows (NT, 95, 98, and 2000)

Novell LDAP JDBC Driver

The Novell LDAP JDBC Driver is a Java database connectivity driver that enables Java programs to execute SQL (Structured Query Language) statements to access Novell Directory Services® (NDS).

Novell Controls for ActiveX

You can use the Internet Directory Query Control to query and retrieve information about directory entries from LDAP-based directories.

LDAP and NDS Terminology

The following sections describe some of the terminology and conceptual differences between LDAP and NDS:

- ♦ “Objects and Entries, Attributes and Properties” on page 14
- ♦ “Naming Conventions” on page 14
- ♦ “LDAP Naming Contexts and NDS Partitions” on page 15
- ♦ “Access Control and Effective Rights” on page 16

Objects and Entries, Attributes and Properties

In LDAP documentation, an entry consistently means a record in the directory database. In NDS documentation, such a record is fairly consistently called an object. Since object becomes ambiguous when describing object-oriented programming languages, NDS developer documentation is beginning to use the LDAP term, entries. However, some functions, attribute names, and class names use object as part of the name. Novell product documentation for NDS utilities and applications continue to use the NDS term, objects.

In most directory documentation, attributes refer to the fields of a record. LDAP and X.500 conform to this standard. However, a lot of LDAP documentation also uses the word attribute to refer to the attribute’s value. NDS programming documentation uses attribute. However, Novell product documentation for NDS utilities and applications use the term properties to describe fields in a record. The following table highlights some of these differences:

LDAP Term	NDS Developer Term	NDS Product Term	Meaning
Entry	Both Object and Entry	Object	A record in a database
Attribute	Attribute	Property	A field in a database. The value of the field.

Naming Conventions

Distinguished Names.NDS and LDAP support the same character set for entry names and their distinguished names. However, NDS supports a number of styles for rendering distinguished names. For example, a DN for an NDS function can

- ♦ Be typeful or typeless
CN=Kim.OU=Eng.O=Novell *or* Kim.Eng.Novell

- ♦ Contain dots or slashes as the delimiter.
- ♦ Use Unicode or the code-page character set of the computer

LDAP supports one convention: typeful names with a comma delimiter and code-page characters.

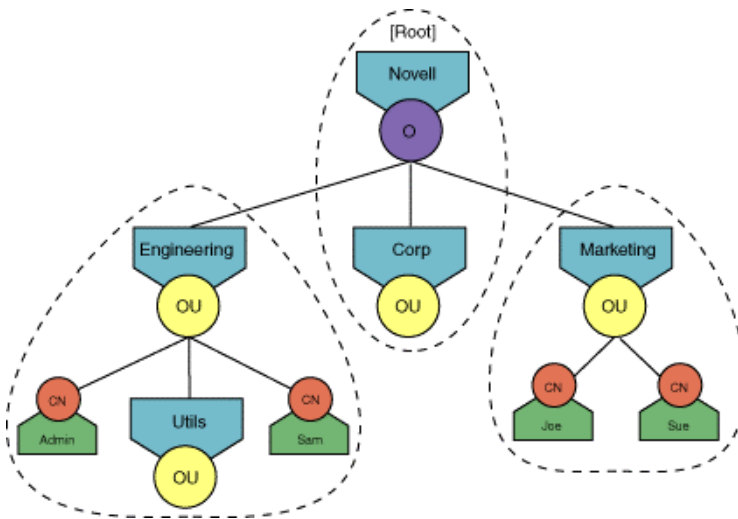
"CN=Kim, OU=Eng, O=Novell"

When using LDAP to access NDS, distinguished names must always be typeful and in LDAP format.

Schema Names. NDS and LDAP support different character sets for object class definitions and attribute definitions. For more information, see “Schema Naming Rules” on page 32.

LDAP Naming Contexts and NDS Partitions

In LDAP, a naming context is the same thing as an NDS partition. Just as an NDS partition is a branch of the NDS tree with only one parent, an LDAP naming context specifies a branch of a hierarchical tree. The following figure illustrates a simple hierarchical tree with three naming contexts ("O=Novell"; "OU=Engineering, O=Novell"; and "OU=Marketing, O=Novell").



Just as with NDS partitions, the naming context is named by the distinguished name of the root container.

Access Control and Effective Rights

Access control determines who has rights to entry information in a directory. In an NDS tree, every entry has an ACL (access control list) attribute which contains the explicit trustee assignments that have been made to the entry and its attributes. In addition, NDS allows rights to be inherited, so that an assignment in a parent container can allow additional trustees to have access to an entry. Functions that calculate effective rights gather information from these parent containers as well as from the ACL attribute. LDAP queries for NDS effective rights return the explicit assignments as well as the inherited rights. Directories that do not allow the inheritance of rights implement the functions to return only explicit trustee assignments.

Containment in Flat and Hierarchical Directories

Flat directories allow an entry to be added without specifying a location. Hierarchical directories require a location to be specified, and the purpose of the distinguished name is to fully describe the location of the entry. However, not all hierarchical directories have rules about where entries can be located. Many directories, such as NDS, require a location and enforce rules about where classes of entries can be located in the directory. For example, user or inetOrgPerson entries cannot be created in a Country container. Directories without such rules allow entries to be created any where in the tree. Sometimes these directories give the appearance of hierarchy, but in reality place all entries in flat database with a link to the entry’s assigned parent.

Novell LDAP Servers

Novell has provided an LDAP server for accessing the NDS directory since the release of NetWare 4.1. The LDAP server in NetWare 5 added support for LDAP v3. The following table compares the features of the newest release of the LDAP server in NDS eDirectory 8.5 with previous releases of the LDAP server.

LDAP Feature	NDS eDirectory	NDS eDirectory	NDS 8	NDS
	Verison 8.5	Version 8.3x	Version 8.1x	Version 7.xx
Authentication (anonymous, clear text, and SSL)	Yes	Yes	Yes	Yes
Mutual authentication	Yes	No	No	No

LDAP Feature	NDS eDirectory	NDS eDirectory	NDS 8	NDS
	Version 8.5	Version 8.3x	Version 8.1x	Version 7.xx
Configure port for clear text and SSL binds	Yes	No	No	No
Enforcement of connection management policies (concurrent connections and time restrictions)	Yes	No	No	No
Enforcement of password restrictions (length, grace logins, expiration, and uniqueness)	Yes	No	No	No
Entry management (search, modify, compare, rename, adding, deleting)	Yes	Yes	Yes	Yes
Naming context (partition) and replica management	Yes	No	No	No
Thread support	Yes	Yes	Yes	Yes
Virtual list view control	Yes	Yes	Yes	No
Server-side sort controls	Yes	Yes	Yes	No
Set indexes for faster searching	Yes	No	No	No
LDAP extensions	Yes	Yes	No	No
Readable root DSE	Yes	Yes	Yes	Yes
Referrals and traversals	Yes	Yes	Yes	Restrictions
Read and write schema	Yes	Yes	Read—Yes	No
Modify existing schema definitions	Yes	No	No	No
Auxiliary classes	Yes	Yes	Yes	No
Valid LDAP names require no mapping	Yes	Yes	No	No

LDAP Feature	NDS eDirectory	NDS eDirectory	NDS 8	NDS
	Verison 8.5	Version 8.3x	Version 8.1x	Version 7.xx
Generated LDAP name for all NDS names which are not mapped or which are not valid	Yes	No	No	No
Access to NDS compound syntaxes	Yes	Selected	No	No
Operational attributes	Yes	No	No	No
Refresh LDAP server from LDAP	Yes	No (only from NDAP))	No (only from NDAP)	No (only from NDAP)

For an application to use the LDAP features in NDS 8 or eDirectory, the application must attach to an LDAP server running that version of NDS. In addition, the data must be local to the LDAP server if the application wants to use the sort and virtual list controls. To use these controls, the LDAP server must hold replicas of the partitions that contain the information you want to sort. In NDS eDirectory 8.5, you can configure filtered replicas to contain just the information you want to sort.

LDAP Support for NDS Features

LDAP supports access to the following NDS features:

- ♦ Authentication. LDAP authentication includes anonymous binds, clear text binds, and SSL binds. For NDS, these LDAP authentication methods mean the following:
 - ♦ An anonymous bind is an unauthenticated connection with [Public] access to the directory.
 - ♦ A clear text bind is an authentication over an unencrypted channel. The client sends a user name and a clear-text password. The LDAP server must be configured to accept unencrypted passwords.
 - ♦ An SSL bind is an authentication over an encrypted channel. All data, including the password, is encrypted. NDS clients have access to SSL binds only through LDAP.
- ♦ Adding, modifying, and deleting entries and attributes in the directory.
- ♦ Reading, sorting, and searching entries and attributes in the directory.

- ♦ Reading schema definitions (object classes and attributes).
- ♦ Adding and deleting schema definitions (object classes and attributes).
The LDAP server in NDS eDirectory 8.5 supports the modifying of class definitions and attribute definitions as long as the modifications increase functionality rather than restrict it.

The LDAP protocol does not yet support access to replication, partition, and synchronization services. These services require LDAP extensions that have been developed for NDS eDirectory 8.5 (for more information, see “LDAP Extensions” on page 22).

LDAP Controls

NDS as a directory provides more services than are available with the standard LDAP functions or Java classes. LDAP v3 provides a method for extending its functionality through LDAP controls and extensions. The LDAP server in NDS eDirectory contains a registration mechanism for LDAP extensions. In future releases, Novell will write LDAP extensions to access such features as ACLs, replication, partition, and synchronization services. Novell is currently working with standards bodies to define ways to access such services.

NDS supports the following controls in NDS 8 and above.

OID	Description
1.2.840.113556.1.4.473	Sever-side sort control request
1.2.840.113556.1.4.474	Server-side sort control response
2.16.840.1.113730.3.4.9	Virtual list view request
2.16.840.1.113730.3.4.10	Virtual list view response

NDS eDirectory 8.5 supports these listed controls, but it does not advertise its support.

NOTE: NDS eDirectory does not currently support persistent search.

For more information about the controls, see the following:

- ♦ “Server-Side Sorting and Sort Keys” on page 20
- ♦ “Virtual List Views” on page 21

Server-Side Sorting and Sort Keys

When using server-side sorting with sort keys, be aware of the following NDS imposed limitations:

- ♦ Server-side sorting is not supported on NDS versions 7.x and below. It is supported only on NDS versions 8 or higher.
- ♦ Reverse sorting is not supported.
- ♦ The data for the subtree being searched must be local to the LDAP server.
- ♦ If an attribute has multiple values, duplicate entries are returned. For example, if a User entry has John and Jack as CN values, then this entry is returned in sorted entries for both Jack and John.
- ♦ NDS 8 allows only one attribute to be selected for sorting; thus your search filter can contain only one attribute.
- ♦ NDS eDirectory 8.5 allows system administrators to define indexes for attributes. NDS versions 8.1x through 8.3x come with predefined indexes for a few common attributes.

If an attribute specified in the search filter is not indexed and the sort control is flagged critical, the LDAP server returns an error. If the control is not flagged critical and the attribute has no index, the LDAP server ignores the sort control. See the table below for a list of indexed attributes defined for NDS 8 and above.

LDAP Name	NDS Name	Description
cn	CN	Sorts entries by their cn attribute. In NDS, most leaf entries use the cn attribute for their naming attributes, whereas most container entries use other attributes for naming.
givenName	Given Name	Sorts entries by their givenName attribute.
surname sn	Surname	Sorts entries by their surname attribute.

LDAP Name	NDS Name	Description
uid	uniqueID	Sorts entries by their uniqueID attribute (LDAP's uid attribute). This is not a required NDS attribute. Unless a utility has been run that creates these for the entries in the NDS tree, this index may contain no entries.

Two of the indexes, cn and uid, contain substring indexes on their values. Substring indexes contain multiple keys for each value so that searching for a specific substring of a value is optimized. For example, a search string of

`*ary`

would quickly find the values

Hillary
Mary
Scary

The index can find these values without comparing the substring value of “ary” with every string in the index.

Virtual List Views

When accessing the lists created with the sort control, be aware of the following NDS implementation details:

- ♦ The sort control in the LDAP's operation control list must be before the virtual list control.
- ♦ Entries that have an attribute with multiple values are returned multiple times in the list (see “Server-Side Sorting and Sort Keys” on page 20).
- ♦ Entries that have no value for the attribute are not returned.
- ♦ The data for the subtree being searched must reside on the LDAP server. The LDAP server will not follow a referral to another NDS server to retrieve the data.

In NDS, the index number for an entry in the list is not assigned, but calculated each time a request to view that entry is made. Therefore, repeated calls to view an entry using an index number can return different entries.

LDAP Extensions

Novell has added LDAP extensions so that LDAP clients can manage naming contexts and replicas. These extensions allow LDAP clients to manage the following:

- ♦ Split and join naming contexts
- ♦ Add, delete, list, and change replicas
- ♦ Synchronize replicas and the schema

NDS supports the following extensions in NDS eDirectory 8.5.

OID	Name
2.16.840.1.113719.1.27.100.1	ndsToLdapResponse
2.16.840.1.113719.1.27.100.2	ndsToLdapRequest
2.16.840.1.113719.1.27.100.3	createNamingContextRequest
2.16.840.1.113719.1.27.100.4	createNamingContextResponse
2.16.840.1.113719.1.27.100.5	mergeNamingContextRequest
2.16.840.1.113719.1.27.100.6	mergeNamingContextResponse
2.16.840.1.113719.1.27.100.7	addReplicaRequest
2.16.840.1.113719.1.27.100.8	addReplicaResponse
2.16.840.1.113719.1.27.100.9	refreshLDAPServerRequest
2.16.840.1.113719.1.27.100.10	refreshLDAPServerResponse
2.16.840.1.113719.1.27.100.11	removeReplicaRequest
2.16.840.1.113719.1.27.100.12	removeReplicaResponse
2.16.840.1.113719.1.27.100.13	namingContextEntryCountRequest
2.16.840.1.113719.1.27.100.14	namingContextEntryCountResponse
2.16.840.1.113719.1.27.100.15	changeReplicaTypeRequest
2.16.840.1.113719.1.27.100.16	changeReplicaTypeResponse
2.16.840.1.113719.1.27.100.17	getReplicaInfoRequest

OID	Name
2.16.840.1.113719.1.27.100.18	getReplicaInfoResponse
2.16.840.1.113719.1.27.100.19	listReplicaRequest
2.16.840.1.113719.1.27.100.20	listReplicaResponse
2.16.840.1.113719.1.27.100.21	receiveAllUpdatesRequest
2.16.840.1.113719.1.27.100.22	receiveAllUpdatesResponse
2.16.840.1.113719.1.27.100.23	sendAllUpdatesRequest
2.16.840.1.113719.1.27.100.24	sendAllUpdatesResponse
2.16.840.1.113719.1.27.100.25	requestNamingContextSyncRequest
2.16.840.1.113719.1.27.100.26	requestNamingContextSyncResponse
2.16.840.1.113719.1.27.100.27	requestSchemaSyncRequest
2.16.840.1.113719.1.27.100.28	requestSchemaSyncResponse
2.16.840.1.113719.1.27.100.29	abortNamingContextOperationRequest
2.16.840.1.113719.1.27.100.30	abortNamingContextOperationResponse
2.16.840.1.113719.1.27.100.31	getContextIdentityNameRequest
2.16.840.1.113719.1.27.100.32	getContextIdentityNameResponse
2.16.840.1.113719.1.27.100.33	getEffectivePrivilegesRequest
2.16.840.1.113719.1.27.100.34	getEffectivePrivilegesResponse

The Novell Import Convert Export utility uses the following extensions. They are not general extensions designed for developer use but are designed to support the LDAP Bulk Update Replication Protocol (LBURP).

OID	Name
2.16.840.1.113719.1.142.100.1	startFramedProtocolRequest
2.16.840.1.113719.1.142.100.2	startFramedProtocolResponse
2.16.840.1.113719.1.142.100.4	endFramedProtocolRequest

OID	Name
2.16.840.1.113719.1.142.100.5	endFramedProtocolResponse
2.16.840.1.113719.1.142.100.6	lburpOperationRequest
2.16.840.1.113719.1.142.100.7	lburpOperationResponse

A query to the LDAP server's root DSE returns the supported extensions.

Differences Among LDAP Servers

You can use LDAP to implement an application that works only on a specific type of LDAP server with a specific type of directory. No two LDAP servers and their directories function exactly alike. For example, the LDAP servers available from Novell, Netscape, IBM, and Oracle all function slightly differently. If you want your application to be generic and work with most LDAP servers and their directories, you need to be aware of differences and the causes of those differences.

The following guidelines are not a definitive list of the differences between Novell and other implementations of an LDAP server and directory. These are intended to help make you code your application so that it will work on more than one directory. The definitive list will come as you test your application against multiple directories. Our suggestions are divided into the following topics:

- ♦ “Schema” on page 24
- ♦ “Validation and Security Checking” on page 25
- ♦ “Directory Structure” on page 26
- ♦ “LDAP Control Support” on page 27
- ♦ “LDAP Extension Support” on page 28

Schema

No two directory schemas contain the same class and attribute definitions. Be aware of the following schema operations.

Guideline: Never assume that a particular attribute is available in a directory's schema.

The application should first read the schema definition before trying to use an attribute. For example, some non-standard LDAP attributes have not been added to the NDS schema. Therefore, if you code your application to expect a particular attribute, the application could fail.

Guideline: When extending a schema, select the most flexible method.

In some schemas, the Person-type class definitions allows an entry to be a container object. In the NDS schema, person-type definitions allow only leaf entries. The schema specification for LDAP v3 includes support for auxiliary classes. If you select to add attributes to a Person-type entry through an auxiliary class rather than as objects subordinate to the Person class, the application can work with both LDAP servers.

Guideline: Order schema extensions to accommodate referential validation.

If you define a class that references specific attributes, NDS verifies that all referenced attributes exist before it allows the class to be added. Therefore, when extending the schema, you must add the attributes before adding the class. Other directories perform no referential validation. To accommodate both types of directories, always add attribute definitions before adding class definitions.

Validation and Security Checking

LDAP servers have different rules for processing data. Be aware of the following.

Guideline: Order operations to accommodate integrity checking.

The Novell LDAP server uses NDS security and supports NDS validity and integrity checking. NDS verifies all attribute values that are DN's, a feature that increases the security and integrity of the directory. For example, when the DN of a user is added to a group membership list, NDS verifies that this user exists in the directory and creates a reference from the user to the group. This reference is used to remove the user from the group when the user is deleted from the directory. The delete operation removes all references to that user. If the system administrator creates another user with the same name,

there is no residual information from the old user for the new user to connect with.

Some directories do not do this type of verification, and without the implementation of a reference, the user can be deleted from the directory while the user's name remains in a group membership list. If the system administrator fails to manually remove the user from the group list and then later adds another user with the same name, that new user becomes a member of the group the old user belonged to, regardless of the system administrator's intentions.

In directories that do not verify DNs, users and groups can be added to the directory in any order. You can add the groups with their membership lists and then the users that are associated with the groups. NDS will not allow this. The users must be added first, and then the groups with their membership lists. You can make your application work with both types of directories if you order the operation to accommodate NDS integrity checking.

Guideline: Request an LDAP v3 binding when the LDAP specification requires it.

Novell has the overall philosophy of restricting rights and enforcing those restrictions. According to the LDAP specifications, clients should have an LDAP v3 bind in order to read the root DSE object. Novell's LDAP server enforces this restriction. Not all LDAP servers enforce this restriction. To make your application work with most LDAP servers, you need to request an LDAP v3 bind when reading the root DSE.

Directory Structure

LDAP directories vary in structure and structure rules. Be aware of the following.

Guideline: Do not assume that all directories have the same containment structure and naming rules.

Some LDAP servers access a directory with a flat database structure on a single computer. Novell's LDAP server accesses NDS, which has a hierarchical database that is distributed on multiple computers.

In maintaining the hierarchical structure, NDS enforces

- ♦ Containment rules that determine which types of objects another object can contain

- ♦ Naming rules that determine which attributes can be used to name objects

Some LDAP directories, which claim to have containment and naming rules, do not enforce these rules. Therefore, do not build your application with the assumption that a certain entry must use a specific attribute for naming or with the assumption that certain entries can be located anywhere in a directory or only in one type of container. Applications built with such assumptions will not work with all LDAP directories.

Guideline: Build your application to handle referrals.

NDS is a distributed database, which means that all available data is not usually local to an LDAP server. In most NDS trees, the data is stored in replicas on multiple servers. Therefore, the data that is local is the data that is in the replicas stored on the LDAP server, and the rest of the data is remote.

Because NDS has been designed to be distributed and to hide that fact from most end users, the LDAP server can be configured to search automatically for the data in remote locations. Such a search takes longer than a local search that returns referrals. To remove the impression that Novell's LDAP server is slow, the default configuration for the LDAP server in NDS eDirectory has been changed. It is now configured as most other LDAP servers, to return a referral when the data isn't local. Your application should be designed to handle referrals.

LDAP Control Support

Guideline: If your application uses LDAP controls, use only those features that all LDAP servers support.

LDAP controls have not been implemented alike. Each LDAP server has its own peculiarities and restrictions for sorted controls and virtual list views. See the documentation for each LDAP server you want your application to run on for a list of those restrictions. For the implementation details on Novell's LDAP server, see "Server-Side Sorting and Sort Keys" on page 20 and "Virtual List Views" on page 21.

LDAP Extension Support

Guideline: If your application uses LDAP extensions, query the LDAP server to get the supported extensions.

LDAP extensions are specific to an LDAP server. For example, each LDAP server in an NDS could support different extensions. LDAP servers from different vendors will support different extensions. Currently, all LDAP extensions are in a draft state with the IETF and none has had the time to move from proposal to acceptance as a standard. Novell is proposing its naming context extensions to IETF.

LDAP Search Filters

The LDAP search filter grammar is specified in RFC 2254 and 2251. The grammar uses ABNF notation.

```
filter = " ( " filtercomp " ) "  
filtercomp = and / or /not /item  
  
and = "&" filterlist  
    filterlist = 1*filter  
  
or = "|" filterlist  
    filterlist = 1*filter  
  
not = "!" filterlist  
    filterlist = 1*filter  
  
item = simple/present/substring/extendible  
  
simple = attr filtertype value  
    attr = name | name;binary  
    filtertype = equal/approx/greater/less  
    value = data valid for the attribute's syntax  
  
equal = "="  
approx = "~="  
greater = ">="  
less = "<="  
  
present = attr "=*"  
    attr = name | name;binary  
  
substing = attr "=" [initial] any [final]
```

```

attr = name | name;binary
initial = value
any = "*" *(value "*")
final = value

extensible = attr [":dn"] [":" matchingrule] ":=value
              / [":dn"] ":" matchingrule ":= value
              / matchingrule = name | OID

```

For additional options for the attr option, see Section 4.1.5 of RFC 2251.

For additional information on the value option, see Section 4.1.6 of RFC 2251.

IMPORTANT: NDS eDirectory processes an LDAP approximate (~=) operator as if it were an equal (=) operator. NDS does not currently support extensible matching rules.

The following examples illustrate a few search filters using this grammar:

```

(cn = Kim Smith)
(!(cn = Chris Jones))
(&(objectClass=inetOrgPerson) (| (sn=Smith) (cn=Chris S*) ) )

```

Connecting and Authenticating to NDS with LDAP

The LDAP connection and authentication model is a two-step process.

- 1** The client first establishes a connection to the LDAP server.
 - ♦ Non-secure, clear-text connections are established by default on port 389.
 - ♦ Data encrypted connections are established by default on port 636.

The port numbers are configurable and may vary from LDAP server to LDAP server.

- 2** After the connection is established, the client then binds with the appropriate credentials for the port. The bind authenticates the client to NDS and establishes an identity.

The Novell LDAP server listens on both ports for connections and can be configured to accept both types of connections.

Connections on Port 389

LDAP supports various types of binds on port 389:

- ♦ Anonymous binds require the client to send in an empty string for both the user name and the password. The Novell LDAP server accepts such a bind request and establishes the client as the NDS entry [Public].
- ♦ Clear-text password binds require a valid distinguished name and a password. Since clear-text passwords compromise security, NDS by default accepts only encrypted passwords and must be configured to accept clear-text passwords. The client is authenticated with the identity of the distinguished name and given all of its NDS rights.

SSL Connections on Port 636

The SSL init functions exchange encryption keys as part of the connection process, and these keys are then used to encrypt all data sent across the connection.

The NDS server currently supports only server-side authentication. This means that the client must obtain a certificate file from each LDAP server to which the client will authenticate. Each Novell LDAP server must be configured for SSL authentication and export a certificate for clients to use.

The client sends this certificate with an SSL init function. The client then binds using a distinguished name and password, which establishes the client as the NDS identity of the distinguished name. The client can also bind as an anonymous user by sending empty strings for the distinguished name and password. Such a bind request establishes the client as the NDS identity [Public].

Anonymous Binds

An anonymous bind request establishes the client as the NDS identity [Public]. When using the `ldap_get_context_identity_name` function on an anonymous bind, the function returns an empty string rather than [Public].

userPassword Attribute

When creating a user, a value must be set for the `userPassword` attribute in order for the user to log in to the directory.

When using an LDAP compare function to verify a userPassword value, the function can return true and the client can still be locked out of the account. NDS uses the following attributes, not just the userPassword attribute, to control access to an account:

- ♦ Locked By Intruder
- ♦ Login Allowed Time Map
- ♦ Login Disabled
- ♦ Login Expiration Time
- ♦ Login Maximum Simultaneous
- ♦ Password Expiration Interval
- ♦ Password Required

If the password is verified as valid, these other attributes should be checked to determine why the client cannot access the account. See the *NDS Schema Reference* for a description of these attributes.

In NDS eDirectory 8.5, these attributes can be accessed from LDAP by using the name without the spaces. In previous versions of NDS, these attributes must be mapped to valid LDAP names to access them.

LDAP and NDS Schema

Although both the LDAP schema and the NDS schema are based on X.500, some of the implementation details are quite different with each being more restrictive in some areas and less restrictive in other areas. The following sections describe some of these differences:

- ♦ “Schema Naming Rules” on page 32
- ♦ “Schema Mapping” on page 32
- ♦ “LDAP Operational Attributes” on page 33
- ♦ “Attribute Flags” on page 37
- ♦ “Object Class Flags” on page 40
- ♦ “Syntax Definitions” on page 42
- ♦ “Auxiliary Classes versus Modifications to Class Definitions” on page 44

Schema Naming Rules

NDS supports different rules for naming object class definitions and attribute definitions. For example, NDS allows spaces and punctuation characters in the name. These conventions are incompatible with LDAP naming conventions, which restrict the name to alphabetical characters, no spaces, and one punctuation character, the hyphen.

The LDAP server in NDS eDirectory automatically maps the attributes and classes defined by RFC 2256 to their LDAP equivalent names. (For a list, see the LDAP Name index in the *NDS Schema Reference*.) If LDAP clients need access to the other NDS classes and attributes that have incompatible names, the system administrator needs to use ConsoleOne to manually map them to an LDAP compatible name.

Even if the name is compatible with LDAP conventions, an LDAP client may not be able to access the attribute because the attribute's syntax is not supported by the LDAP server. For a list of supported syntax definitions, see "Syntax Definitions" on page 42.

The following paragraphs describe a couple of special cases for object class definitions.

User and inetOrgPerson. By default, inetOrgPerson is mapped to the NDS User object class, and you can access this class using the LDAP names of inetOrgPerson or user. By default, the User class definition does not contain all the standard attributes for inetOrgPerson. To add these attributes to the User class definition, the system administrator must run a schema file (nov_inet.sch). In NDS eDirectory, Novell also supplies a schema file that creates an inetOrgPerson class with the standard LDAP attributes. Your application will need to read the schema and the directory to determine which class contains the users for the system.

residentialPerson. In NDS eDirectory, Novell supplies a schema file that adds the residentialPerson class definition to the schema with the standard LDAP attributes. Your application will need to read the schema to determine if the system administrator has added this class to the schema.

Schema Mapping

Since the NDS schema has many class and attribute definitions with illegal LDAP names, these definitions must be mapped to a legal LDAP name. All LDAP servers in the NDS tree must map the same LDAP class or attribute to

the same NDS class or attribute. Each version of NDS has made this mapping easier:

- ♦ In NDS eDirectory 8.3x, all standard LDAP attribute and class definitions are mapped to the corresponding NDS attribute or class definitions when NDS is installed. The *NDS Schema Reference* contains an index to these LDAP names. If the schema is extended with classes and attributes that use invalid LDAP names, these classes and attributes need to be manually mapped to valid LDAP names.
- ♦ In NDS eDirectory 8.5, all NDS attribute and class definitions can be accessed from LDAP by using the NDS name with the colon and spaces removed. Also, any required mappings can be processed from an LDIF file during installation.

From ConsoleOne, system administrators can add mappings for classes and attributes that are not automatically mapped. The LDAP server needs to be stopped and started to make the mapped attributes and classes visible from LDAP.

LDAP Operational Attributes

In NDS, not all information about an entry is kept in attributes, for example, an entry's base class, last modified time, or creation time. Through NDAP, NDS has made this information available through DSI flags. In NDS eDirectory 8.5, this information is available through LDAP as operational attributes. The information is read-only. Clients cannot modify the attributes.

These attributes are not returned in search results unless explicitly requested by name. NDS eDirectory 8.5 supports the following operational attributes on all entries in the directory.

Operation Attribute	Description
createTimeStamp	Contains when the entry was created. Standard LDAP attribute (RFC 2252)
creatorsName	Contains the distinguished name of the user that created this entry. Standard LDAP attribute (RFC 2252)

Operation Attribute	Description
entryFlags	Contains information about an entry's state, for example whether the entry is an alias, a partition, or a container. NDS-specific attribute
federationBoundary	Contains where the federation boundary begins for a DNS-rooted tree. NDS-specific attribute
localEntryID	Contains the record number for the entry in the server's local database. NDS-specific attribute
modifiersName	Contains the distinguished name of the last user that modified this entry. Standard LDAP attribute (RFC 2252)
modifyTimeStamp	Contains when the entry was last modified. Standard LDAP attribute (RFC 2252)
structuralObjectClass	Contains the base class of the entry. Standard LDAP attribute
subordinateCount	Contains the number of entries immediately subordinate to this entry. NDS-specific attribute
subschemaSubentry	Contains the LDAP name for the schema. Standard LDAP attribute (RFC 2252). For more information, see "Subschema Subentry Attributes" on page 36.

For more information, see LDAP Operational Attributes in the *NDS Schema Reference*.

Root DSE Attributes

The LDAP server maintains the root DSE attributes, and clients can read these attributes but cannot modify them. The Novell LDAP server supports the following attributes.

Attribute	Description	NDS Version
namingContexts	Contains the distinguished names of the naming contexts (or replicas) on the LDAP server. Standard LDAP attribute (RFC 2252).	8.x
altServer	Contains the name of alternative servers if this one is unavailable in subsequent requests Standard LDAP attribute (RFC 2252)..	85
supportedExtension	Contains a list of OIDs that identify supported extended operations. Standard LDAP attribute (RFC 2252).	8.x
supportedControl	Contains a list of OIDs that identify supported controls. Standard LDAP attribute (RFC 2252).	8.x . In version 85, controls are not advertized.
supportedSASLMechanisms	Contains a list of supported SASL security features. Standard LDAP attribute (RFC 2252).	85
supportedLDAPVersion	Contains a list of LDAP versions implemented by the server. Standard LDAP attribute (RFC 2252).	8.x

Attribute	Description	NDS Version
subschemaSubentry	Contains the distinguished name of the subschema subentry which is the name of the schema for this server. For NDS, this is always cn=schema. Standard LDAP attribute (RFC 2252).	8.x
vendorName	Contains a string with the name of the company. NDS-specific attribute.	85
vendorVersion	Contains the LDAP server version. NDS-specific attribute.	85
directoryTreeName	Contains the NDS tree name. NDS-specific attribute.	85
dsaName	Contains the distinguished name of the server. NDS-specific attribute.	85

In a DNS-rooted tree, you can find the federation boundary by one of the following methods:

- ♦ Using the namingContext attribute, search one of the naming contexts for its federationBoundary attribute.
- ♦ Using the dsaName attribute, search the server for its federationBoundary attribute. Use this method only if a naming context is not available.

Subschema Subentry Attributes

Clients must specifically request attribute information about the subschema entry. It is never returned in a generic request to read all information about all

entries. Clients must request a base object search with the search filter set to the following: "objectClass=subschema"

The Novell LDAP server supports the following attributes.

Attribute	Description	NDS Version
cn	Contains the RDN of the subschema entry.	8.x
objectClasses	Contains a value for each object class known by the LDAP server.	8.x
objectClass	Contains the parent object classes of the subschema subentry. It always contains two classes: top and subschema.	8.x
attributeTypes	Contains a value for each attribute definition known by the LDAP server.	8.x
ldapSyntaxes	Contains a value for each syntax definition known by the LDAP server.	85

Clients add attribute definitions and object classes to the schema by adding values to the attributeTypes and objectClasses attributes of the subentry.

Attribute Flags

NDS supports numerous attribute definition flags that affect the type of data an attribute can contain and that control its synchronization schedule. The LDAP server in NDS eDirectory can set and get information about the following NDS extended flags. By default, when creating an attribute definition, none of these flags are set. To obtain the information when reading the schema, you must send the flag with the request to read the attribute.

Extended LDAP Flags	Description
X-NDS_PUBLIC_READ	When set, allows anyone to read the attribute's value even though such rights have not been granted or inherited. Using this flag makes access to the attribute extremely efficient because NDS performs no rights checking. When not set, users must have been granted rights or inherit rights to read the attribute's value. In NDAP, this is equivalent to the DS_PUBLIC_READ flag set to True.
X-NDS_SERVER_READ	When set, allows the NCP Server object to read the attribute's value even though such rights have not been granted or inherited. When not set, the NCP Server object must be granted rights or inherit rights to read the attribute's value. In NDAP, this is equivalent to the DS_SERVER_READ flag set to True.
X-NDS_NEVER_SYNC	When set, prevents changes to this attribute from synchronizing with other replicas. The information in the attribute is specific to the replica. When not set, changes to the attribute are synchronized to other replicas. In NDAP, this is equivalent to the DS_PER_REPLICA flag set to True.
X-NDS_NOT_SCHED_SYNC_IMMEDIATE	When set, allows the attribute's value to change without scheduling synchronization, and synchronization will start within 30 minutes. When not set, causes any changes to the attribute to schedule immediate synchronization (within 10 seconds). In NDAP, this is equivalent to the DS_SYNC_IMMEDIATE flag set to False.

Extended LDAP Flags	Description
X-NDS_SCHED_SYNC_NEVER	When set, allows the attribute's value to change without scheduling synchronization. The attribute can wait until the next scheduled synchronization cycle to propagate its changes. When not set, causes any changes to the attribute to schedule synchronization. Developers can only read this flag. In NDAP, this is equivalent to the DS_SCHEDULE_SYNC_NEVER flag set to True.
X-NDS_LOWER_BOUND	When set, specifies the lower boundary for a string or integer syntax. When not set, the attribute has no lower boundary. In NDAP, this is equivalent to the DS_SIZED_ATTR flag set to True
X-NDS_NAME_VALUE_ACCESS	This flag only works on attributes which use a DN syntax and contain a list of entries, such as groupMembership. When set, requires users to have supervisor rights to the entry before they can add or delete the entry as a value for this attribute. When not set, requires the user to have read rights to read the values and write rights to modify the values. In NDAP, this is equivalent to the DS_WRITE_MANAGED flag set to True.
X-NDS_NAME	When creating an attribute, specifies the legacy NDS attribute that automatically maps to this LDAP attribute. This is new in NDS eDirectory 8.5 and should be used to make attributes available to previous versions of the LDAP server in an NDS tree. When reading the attribute definition, returns the legacy NDS attribute name.

The standard LDAP attribute flags can also be used. The following table lists the LDAP name and the corresponding NDAP name.

Standard LDAP Flags	NDAP Flag
SINGLE-VALUE	DS_SINGLE_VALUED_ATTR set to True
COLLECTIVE	Not supported
NO-USER-MODIFICATION	DS_READ_ONLY_ATTR set to True
USAGE userApplications	None required. This sets the attribute as a normal attribute. The other USAGE flags can only be set by NDS.
USAGE directoryOperation	DS_OPERATIONAL (set by NDS)
USAGE distributedOperation	DS_OPERATIONAL (set by NDS)
USAGE dSAOperation	DS_OPERATIONAL (set by NDS)

Object Class Flags

NDS uses a set of flags to define allowable class operations. When adding a new object class definition to the schema, you can set the following flags. When reading definitions, you send the flags to obtain the information.

Extended LDAP Flags	Description
X-NDS_NOT_CONTAINER	When set, indicates that this object class cannot contain other entries and is thus a leaf entry. When not set, indicates that this object class can contain other entries and is thus a container class. In NDAP, this is equivalent to the DS_CONTAINER_CLASS flag set to False.

Extended LDAP Flags	Description
X-NDS_CONTAINMENT	When included, this flag is followed by a list of object classes that can be the parent container of the object class that is being defined. When not included, the object class that is being defined is automatically assigned containment classes of country, organization, organizationalUnit, locality, and domain. In NDAP, this is equivalent to the DS_AMBIGUOUS_CONTAINMENT flag set to False.
X-NDS_NAMING	When included, this flag is followed by the list of attributes that can be used to name entries based on this object class definition. When not included, the naming attributes for the object class are all of the MAY and MUST attributes that use a string-type syntax. In NDAP, this is equivalent to the DS_AMBIGUOUS_NAMING flag set to False.
X-NDS_NONREMOVABLE	When set, indicates that the class cannot be removed even if no entries are using the definition. This flag is placed on all classes in the NDS operational schema. NDS 8 and higher allow application developers to set this flag. When not set, indicates that the class can be removed from the schema if no entries are using the definition. In NDAP, this is equivalent to the DS_NONREMOVABLE_CLASS flag set to True.
X-NDS_NAME	When defining an object class, specifies the legacy NDS object class that automatically maps to this LDAP class. This is new in NDS eDirectory 8.5 and should be used to make classes available to previous versions of the LDAP server in an NDS tree. When reading object class definitions, returns the legacy NDS name for this object class.

The standard LDAP class flags can also be used. The following table lists the LDAP name and the corresponding NDAP name.

Standard LDAP Flags	NDAP FLaG
ABSTRACT	DS_EFFECTIVE_CLASS set to False
STRUCTURAL	DS_EFFECTIVE_CLASS set to True
AUXILIARY	DS_AUXILIARY_CLASS set to True

Syntax Definitions

The LDAP server allows LDAP access to NDS attributes if the NDS attribute uses an LDAP compatible syntax. For example, this makes all NDS attributes that use the Case Ignore String syntax available through LDAP because LDAP supports the Case Ignore String syntax. NDS attributes that use a compound syntax (such as the Timestamp syntax with its fields for time, replica number, and event identifier) are not automatically accessible through LDAP. The LDAP server has made most of these syntax definitions available.

The LDAP server supports the following NDS syntax definitions. The NDS column lists the minimum version of NDS for the LDAP server to support the syntax.

NDS Name	LDAP Descriptive Name	OID	NDS
SYN_STREAM	Binary	1.3.6.1.4.1.1466.115.121.1.5	7.xx
SYN_BOOLEAN	Boolean	1.3.6.1.4.1.1466.115.121.1.7	7.xx
SYN_DIST_NAME	DN	1.3.6.1.4.1.1466.115.121.1.12	7.xx
SYN_CI_STRING	Directory String	1.3.6.1.4.1.1466.115.121.1.15	7.xx
SYN_FAX_NUMBER	Facsimile Telephone Number	1.3.6.1.4.1.1466.115.121.1.22	7.xx
SYN_TIME	Generalized Time	1.3.6.1.4.1.1466.115.121.1.24	7.xx
SYN_CE_STRING	IA5 String	1.3.6.1.4.1.1466.115.121.1.26	7.xx
SYN_INTEGER	Integer	1.3.6.1.4.1.1466.115.121.1.27	7.xx
SYN_INTERVAL	Integer	1.3.6.1.4.1.1466.115.121.1.27	7.xx

NDS Name	LDAP Descriptive Name	OID	NDS
SYN_NU_STRING	Numeric String	1.3.6.1.4.1.1466.115.121.1.36	7.xx
SYN_CLASS_NAME	OID	1.3.6.1.4.1.1466.115.121.1.38	7.xx
SYN_OCTET_STRING	Octet String	1.3.6.1.4.1.1466.115.121.1.40	7.xx
SYN_PO_ADDRESS	Postal Address	1.3.6.1.4.1.1466.115.121.1.41	7.xx
SYN_PR_STRING	Printable String	1.3.6.1.4.1.1466.115.121.1.44	7.xx
SYN_TEL_NUMBER	Telephone Number	1.3.6.1.4.1.1466.115.121.1.50	7.xx
SYN_UNKNOWN	Unknown	2.16.840.1.113719.1.1.5.1.0	7.xx
SYN_CI_LIST	Case Ignore List	2.16.840.1.113719.1.1.5.1.6	85.
SYN_NET_ADDRESS	Tagged Data	2.16.840.1.113719.1.1.5.1.12	8.3x
SYN_OCTET_LIST	Octet List	2.16.840.1.113719.1.1.5.1.13	85.
SYN_EMAIL_ADDRESS	Tagged String	2.16.840.1.113719.1.1.5.1.14	85.
SYN_PATH	Tagged Name and String	2.16.840.1.113719.1.1.5.1.15	8.3x
SYN_REPLICA_POINTER	NDS Replica Pointer	2.16.840.1.113719.1.1.5.1.16	85.
SYN_OBJECT_ACL	NDS ACL	2.16.840.1.113719.1.1.5.1.17	8.3x
SYN_TIMESTAMP	NDS Timestamp	2.16.840.1.113719.1.1.5.1.19	85.
SYN_COUNTER	Counter	2.16.840.1.113719.1.1.5.1.22	85.
SYN_BACK_LINK	Tagged Name	2.16.840.1.113719.1.1.5.1.23	85.
SYN_TYPED_NAME	Typed Name	2.16.840.1.113719.1.1.5.1.25	85.

Most of the definitions with Novell OIDs (2.16.840.1.113719) are structured and have multiple components. The LDAP server converts such a syntax to case ignore strings, using dollar (\$) signs to separate fields of the same data type and (#) signs to separate fields of different data types. See *Attribute Syntax Definitions* in the *NDS Schema Reference* for more information.

The SYN_HOLD syntax is not supported through LDAP and is being discontinued in NDS.

Auxiliary Classes versus Modifications to Class Definitions

NDS and LDAP have had very different conventions about modifying existing object class definitions. Once an object class has been defined in the schema,

- ♦ LDAP conventions assume (1) that the attributes for the object class do not change, and (2) that any new attributes will be added through auxiliary classes to the entry rather than to the class definition.
- ♦ NDS conventions allow applications to add new attributes to the class definitions, or with the release of NDS 8, to add new attributes to the entry through auxiliary classes rather than the class definition.

To maintain backwards compatibility with NDS releases prior to NDS 8, most NDS applications are still adding attributes to class definitions rather than to entries through auxiliary classes.

When accessing NDS, you must read the schema to discover all possible attributes for a class definition. If your application accesses more than one NDS tree, it must read the schema from each tree because the chance of the schemas being the same is very small.

Other Sources of LDAP Information

If you are new to LDAP and Novell development, check out the following sources:

- ♦ Novell's LDAP site (<http://developer.novell.com/nds/ndsldap.htm>). This site contains links to generic LDAP information and to specific information about the Novell LDAP server.
- ♦ Novell's NDS/LDAP Services Access Test Site (<http://nldap.com/nldap>). This site contains links to LDAP RFCs and access to a Novell LDAP server for testing your LDAP application.
- ♦ Novell's DeveloperNet University (<http://developer.novell.com/education/index.html>). This site contains source code for specific LDAP operations and the labs take you step-by-step through the process.

2

LDAP Return Codes

This chapter describes the following:

- ♦ Return codes sent by the LDAP client or libraries
- ♦ Return codes sent by the LDAP server
- ♦ Result code structure used by the return codes

LDAP Client Return Codes

The LDAP client or libraries return codes set the following return codes.

Hex	Decimal	Constant: Description
0x51	81	LDAP_SERVER_DOWN: Indicates that the LDAP libraries cannot establish an initial connection with the LDAP server. Either the LDAP server is down or the specified host name or port number is incorrect.
0x52	82	LDAP_LOCAL_ERROR: Indicates that the LDAP client has an error. This is usually a failed dynamic memory allocation error.
0x53	83	LDAP_ENCODING_ERROR: Indicates that the LDAP client encountered errors when encoding an LDAP request intended for the LDAP server.
0x54	84	LDAP_DECODING_ERROR: Indicates that the LDAP client encountered errors when decoding an LDAP response from the LDAP server.
0x55	85	LDAP_TIMEOUT: Indicates that the time limit of the LDAP client was exceeded while waiting for a result.
0x56	86	LDAP_AUTH_UNKNOWN: Indicates that the ldap_bind or ldap_bind_s function was called with an unknown authentication method.

Hex	Decimal	Constant: Description
0x57	87	LDAP_FILTER_ERROR: Indicates that the ldap_search function was called with an invalid search filter.
0x58	88	LDAP_USER_CANCELLED: Indicates that the user cancelled the LDAP operation.
0x59	89	LDAP_PARAM_ERROR: Indicates that an LDAP function was called with an invalid parameter value (for example, the ld parameter is NULL).
0x5A	90	LDAP_NO_MEMORY: Indicates that a dynamic memory allocation function failed when calling an LDAP function.
0x5B	91	LDAP_CONNECT_ERROR: Indicates that the LDAP client has lost either its connection or cannot establish a connection to the LDAP server.
0x5C	92	LDAP_NOT_SUPPORTED: Indicates that the requested functionality is not supported by the client. For example, if the LDAP client is established as an LDAPv2 client, the libraries set this error code when the client requests LDAPv3 functionality.
0x5D	93	LDAP_CONTROL_NOT_FOUND: Indicates that the client requested a control that the libraries cannot find in the list of supported controls sent by the LDAP server.
0x5E	94	LDAP_NO_RESULTS_RETURNED: Indicates that the LDAP server sent no results. When the ldap_parse_result function is called, no result code is included in the server's response.
0x5F	95	LDAP_MORE_RESULTS_TO_RETURN: Indicates that more results are chained in the result message. The libraries set this code when the call to the ldap_parse_result function reveals that additional result codes are available.
0x60	96	LDAP_CLIENT_LOOP: Indicates the LDAP libraries detected a loop. Usually this happens when following referrals.
0x61	97	LDAP_REFERRAL_LIMIT_EXCEEDED: Indicates that the referral exceeds the hop limit. The hop limit determines how many servers the client can hop through to retrieve data. For example, suppose the following conditions: ♦ The hop limit is two. ♦ The referral is to server D which can be contacted only through server B (1 hop) which contacts server C (2 hops) which contacts server D (3 hops) With these conditions, the hop limit is exceeded and the LDAP libraries set this code.

LDAP Server Return Codes

The LDAP server sets the following return codes.

Hex	Decimal	Constant: Description
0x00	0	LDAP_SUCCESS: Indicates the requested client operation completed successfully.
0x01	1	LDAP_OPERATIONS_ERROR: Indicates an internal error. The server is unable to respond with a more specific error and is also unable to properly respond to a request. It does not indicate that the client has sent an erroneous message. In NDS 8.3x through NDS 7.xx, this was the default error for NDS errors that did not map to an LDAP error code. To conform to the new LDAP drafts, NDS 8.5 uses 80 (0x50) for such errors.
0x02	2	LDAP_PROTOCOL_ERROR: Indicates that the server has received an invalid or malformed request from the client.
0x03	3	LDAP_TIMELIMIT_EXCEEDED: Indicates that the operation's time limit specified by either the client or the server has been exceeded. On search operations, incomplete results are returned.
0x04	4	LDAP_SIZELIMIT_EXCEEDED: Indicates that in a search operation, the size limit specified by the client or the server has been exceeded. Incomplete results are returned.
0x05	5	LDAP_COMPARE_FALSE: Does not indicate an error condition. Indicates that the results of a compare operation are false.
0x06	6	LDAP_COMPARE_TRUE: Does not indicate an error condition. Indicates that the results of a compare operation are true.
0x07	7	LDAP_AUTH_METHOD_NOT_SUPPORTED: Indicates that during a bind operation the client requested an authentication method not supported by the LDAP server.
0x08	8	LDAP_STRONG_AUTH_REQUIRED: Indicates one of the following: ♦ In bind requests, the LDAP server accepts only strong authentication. ♦ In a client request, the client requested an operation such as delete that requires strong authentication. ♦ In an unsolicited notice of disconnection, the LDAP server discovers the security protecting the communication between the client and server has unexpectedly failed or been compromised.

Hex	Decimal	Constant: Description
0x09	9	Reserved.
0x0A	10	LDAP_REFERRAL: Does not indicate an error condition. In LDAPv3, indicates that the server does not hold the target entry of the request, but that the servers in the referral field may.
0x0B	11	LDAP_ADMINLIMIT_EXCEEDED: Indicates that an LDAP server limit set by an administrative authority has been exceeded.
0x0C	12	LDAP_UNAVAILABLE_CRITICAL_EXTENSION: Indicates that the LDAP server was unable to satisfy a request because one or more critical extensions were not available. Either the server does not support the control or the control is not appropriate for the operation type.
0x0D	13	LDAP_CONFIDENTIALITY_REQUIRED: Indicates that the session is not protected by a protocol such as Transport Layer Security (TLS), which provides session confidentiality.
0x0E	14	LDAP_SASL_BIND_IN_PROGRESS: Does not indicate an error condition, but indicates that the server is ready for the next step in the process. The client must send the server the same SASL mechanism to continue the process.
0x0F	15	Not used.
0x10	16	LDAP_NO_SUCH_ATTRIBUTE: Indicates that the attribute specified in the modify or compare operation does not exist in the entry.
0x11	17	LDAP_UNDEFINED_TYPE: Indicates that the attribute specified in the modify or add operation does not exist in the LDAP server's schema.
0x12	18	LDAP_INAPPROPRIATE_MATCHING: Indicates that the matching rule specified in the search filter does not match a rule defined for the attribute's syntax.
0x13	19	LDAP_CONSTRAINT_VIOLATION: Indicates that the attribute value specified in a modify, add, or modify DN operation violates constraints placed on the attribute. The constraint can be one of size or content (string only, no binary).
0x14	20	LDAP_TYPE_OR_VALUE_EXISTS: Indicates that the attribute value specified in a modify or add operation already exists as a value for that attribute.
0x15	21	LDAP_INVALID_SYNTAX: Indicates that the attribute value specified in an add, compare, or modify operation is an unrecognized or invalid syntax for the attribute.
	22-31	Not used.

Hex	Decimal	Constant: Description
0x20	32	LDAP_NO_SUCH_OBJECT: Indicates the target object cannot be found. This code is not returned on following operations: ♦ Search operations that find the search base but cannot find any entries that match the search filter. ♦ Bind operations.
0x21	33	LDAP_ALIAS_PROBLEM: Indicates that an error occurred when an alias was dereferenced.
0x22	34	LDAP_INVALID_DN_SYNTAX: Indicates that the syntax of the DN is incorrect. (If the DN syntax is correct, but the LDAP server's structure rules do not permit the operation, the server returns LDAP_UNWILLING_TO_PERFORM.)
0x23	35	LDAP_IS_LEAF: Indicates that the specified operation cannot be performed on a leaf entry. (This code is not currently in the LDAP specifications, but is reserved for this constant.)
0x24	36	LDAP_ALIAS_DEREF_PROBLEM: Indicates that during a search operation, either the client does not have access rights to read the aliased object's name or dereferencing is not allowed.
	37-47	Not used.
0x30	48	LDAP_INAPPROPRIATE_AUTH: Indicates that during a bind operation, the client is attempting to use an authentication method that the client cannot use correctly. For example, either of the following cause this error: ♦ The client returns simple credentials when strong credentials are required. ♦ The client returns a DN and a password for a simple bind when the entry does not have a password defined.
0x31	49	LDAP_INVALID_CREDENTIALS: Indicates that during a bind operation one of the following occurred: ♦ The client passed either an incorrect DN or password. ♦ The password is incorrect because it has expired, intruder detection has locked the account, or some other similar reason.
0x32	50	LDAP_INSUFFICIENT_ACCESS: Indicates that the caller does not have sufficient rights to perform the requested operation.
0x33	51	LDAP_BUSY: Indicates that the LDAP server is too busy to process the client request at this time but if the client waits and resubmits the request, the server may be able to process it then.

Hex	Decimal	Constant: Description
0x34	52	LDAP_UNAVAILABLE: Indicates that the LDAP server cannot process the client's bind request, usually because it is shutting down.
0x35	53	LDAP_UNWILLING_TO_PERFORM: Indicates that the LDAP server cannot process the request because of server-defined restrictions. This error is returned for the following reasons: ♦ The add entry request violates the server's structure rules. ♦ The modify attribute request specifies attributes that users cannot modify. ♦ Password restrictions prevent the action. ♦ Connection restrictions prevent the action.
0x36	54	LDAP_LOOP_DETECT: Indicates that the client discovered an alias or referral loop, and is thus unable to complete this request.
	55-63	Not used.
0x40	64	LDAP_NAMING_VIOLATION: Indicates that the add or modify DN operation violates the schema's structure rules. For example, ♦ The request places the entry subordinate to an alias. ♦ The request places the entry subordinate to a container that is forbidden by the containment rules. ♦ The RDN for the entry uses a forbidden attribute type.
0x41	65	LDAP_OBJECT_CLASS_VIOLATION: Indicates that the add, modify, or modify DN operation violates the object class rules for the entry. For example, the following types of request return this error: ♦ The add or modify operation tries to add an entry without a value for a required attribute. ♦ The add or modify operation tries to add an entry with a value for an attribute which the class definition does not contain. ♦ The modify operation tries to remove a required attribute without removing the auxiliary class that defines the attribute as required.
0x42	66	LDAP_NOT_ALLOWED_ON_NONLEAF: Indicates that the requested operation is permitted only on leaf entries. For example, the following types of requests return this error: ♦ The client requests a delete operation on a parent entry. ♦ The client request a modify DN operation on a parent entry.

Hex	Decimal	Constant: Description
0x43	67	LDAP_NOT_ALLOWED_ON_RDN: Indicates that the modify operation attempted to remove an attribute value that forms the entry's relative distinguished name.
0x44	68	LDAP_ALREADY_EXISTS: Indicates that the add operation attempted to add an entry that already exists, or that the modify operation attempted to rename an entry to the name of an entry that already exists.
0x45	69	LDAP_NO_OBJECT_CLASS_MODS: Indicates that the modify operation attempted to modify the structure rules of an object class.
0x46	70	LDAP_RESULTS_TOO_LARGE: Reserved for CLDAP.
0x47	71	LDAP_AFFECTS_MULTIPLE_DSAS: Indicates that the modify DN operation moves the entry from one LDAP server to another and thus requires more than one LDAP server.
	72-79	Not used.
0x50	80	LDAP_OTHER: Indicates an unknown error condition. This is the default value for NDS error codes which do not map to other LDAP error codes.

LDAP Result Code Structure

An LDAP result code has the following format:

```
LDAPResult ::= SEQUENCE {
    resultCode      ENUMERATED,
    matchedDN       LDAPDN,
    errorMessage    LDAPString,
    referral        [3] Referral OPTIONAL
}
```

These fields contain the following information.

resultCode

Set to the return code by the LDAP server or the LDAP libraries.

matchedDN

Set to the lowest entry (object or entry) in the directory that was matched when the resultCode field is set to LDAP_NO_SUCH_OBJECT, LDAP_ALIAS_PROBLEM, LDAP_INVALID_DN_SYNTAX, or

LDAP_ALIAS_DEREF_PROBLEM. If the resultCode field is set to other result codes, this field is set to a zero-length string.

For example, if the resultCode field is set to LDAP_NO_SUCH_OBJECT, and if the name "CN=Chris, OU=LDAP, O=Novell" is passed in, and if the LDAP server could find the Novell and LDAP entries but not Chris, then this field is set to "OU=LDAP, O=Novell".

errorMessage

Set to a human-readable string that explains the error code.

referral

Optional field that can be set to the URLs of servers that may hold the target entry.

LDAP result codes are returned with each message from the server in a BER-encoded form to the caller of the LDAP function. To retrieve the return code information from a result, use the ldap_parse_result function.

Revision History

The following table lists all changes made to the LDAP and NDS documentation.

June 2001	Added eCommerce Beans to “Supported Platforms” on page 12.
July 2000	Added LDAP operational attributes, OIDs for LDAP extensions, and other NDS eDirectory 8.5 information.
May 2000	Added names, OIDs, NDS version for all NDS syntax definitions.

